Week 1

Constraint Satisfaction Problems: A Formal Definition

◎ Learning Objectives

By the end of this chapter, you will be able to:

- (**H1**) define a Constraint Satisfaction Problem formally in terms of variables, domains and constraints; state what counts as a solution; and distinguish a CSP from a constraint optimization problem (COP) and from a Boolean satisfiability (SAT) instance;
- (**H2**) explain why modeling is a skill rather than a mechanical transcription step, and apply the ideas of viewpoint choice, auxiliary variables and implied (redundant) constraints when building a model;
- (**H3**) formulate classic combinatorial problems – N-queens, Sudoku, map coloring, magic square – as CSPs on paper, stating their variables, domains and constraints explicitly;
- (**H4**) build and run a first working model in OR-Tools CP-SAT: declare variables and domains, add an `AllDifferent` (or pairwise inequality) constraint, invoke the solver, and enumerate all solutions to a small instance;
- (**H5**) translate a paper CSP model into a verified, running CP-SAT program and interpret the solver's output against the formal definition – closing the loop from theory to a working solver run.

Before this course reaches for a single solver, it needs a shared, precise vocabulary for the kind of problem that solver is built to attack. This chapter gives you that vocabulary: a constraint satisfaction problem, defined formally as a triple of variables, domains and constraints; the modeling choices – viewpoint, auxiliary variables, implied constraints – that separate a merely correct model from a good one; the field's classic benchmark problems, formulated on paper as CSPs; and, finally, the exact OR-Tools CP-SAT building blocks that turn a paper model into a program a computer can run. This is also the book's opening chapter, so where a concept needs a prior anchor, this chapter points to your own prerequisites – discrete mathematics, basic graph vocabulary, and Python – rather than to an earlier week, since none exists yet.

Four units carry this chapter. Unit 1 gives the formal triple and contrasts a CSP

with its two closest relatives, constraint optimization and Boolean satisfiability. Unit 2 – this book's anchor unit for tone and pacing – explains why building a good model is a skill you practice, through three concrete levers: viewpoint, auxiliary variables and implied constraints. Unit 3 applies the formal triple and Unit 2's levers to four classic benchmark problems, on paper: N-queens, Sudoku, map coloring and magic square. Unit 4 turns three of those four paper models directly into running OR-Tools CP-SAT code, closing the loop from formal definition to a verified solver run.

This chapter deliberately stays out of several topics that later weeks own outright. It says nothing about *how* a solver actually finds a solution – no propagation mechanics, no search-tree detail – treating the solver as a black box throughout; Week 2 opens that box. It adds no optimization objective to any model; every problem here asks for a solution or all solutions, never the best one, and constraint optimization problems return properly in Week 8. And it uses OR-Tools CP-SAT exclusively – no MiniZinc, no Gurobi, no MIP formulation appear until Weeks 6 and 7.

1.1 What a CSP Is

1 • Pre-instruction **Why It Matters**

You have solved a constraint satisfaction problem before without ever calling it one. Every time you built a class schedule that avoided double-booking a room, filled in a Sudoku grid, or arranged seating so that no two people who cannot stand each other sit together, you were choosing values for unknowns under restrictions. This week gives that everyday pattern a name, a shared vocabulary, and – for the rest of the course – a machine that solves it for you.

1 • Pre-instruction **Prior Knowledge**

No previous week exists in this book; Week 1 is the opening chapter. The reference point instead is what your undergraduate studies already gave you: sets and relations from discrete mathematics – a domain is nothing more than a finite set, and a constraint is nothing more than a relation over a Cartesian product of domains – and basic graph vocabulary, since a map's "which regions touch" structure is a graph you will keep reusing all semester.

This course is called *Constraint Programming*, so it is worth being precise, from the very first page, about the object you are programming with. That object is called a constraint satisfaction problem, and this unit gives it a formal definition before you see a single line of solver code.

1.1.1 The formal triple: variables, domains, constraints

Think about what you actually do when you fill in a class schedule. There are unknowns you must pin down (which room does each class use, at which time), each unknown can only take a handful of legal values (the rooms that exist, the time slots that exist), and there are rules about which combinations of those choices are acceptable (two classes cannot use the same room at the same time). Strip away the scheduling language and you are left with exactly three ingredients, and every constraint satisfaction problem is built out of only these three.

Constraint Satisfaction Problem (CSP): A problem described by a set of variables, a domain of possible values for each, and a set of constraints restricting which combinations of values are jointly allowed.

A **Constraint Satisfaction Problem (CSP)** is defined by exactly the three ingredients the class-schedule example above already hinted at, and nothing more. Every CSP you meet this semester – however large, however specialised its problem domain – reduces to the same three parts below; once you can name them for a given problem, you have already done the hardest conceptual step of modeling it.

- a set of **variables** – the unknowns you must decide, written $X_1, X_2, \ldots, X_n$;
- a **domain** D_i for each variable X_i – the finite set of values that variable is allowed to take; and
- a set of **constraints** – restrictions on which combinations of values, taken together across some subset of the variables, are jointly allowed.

variables: An unknown in a CSP that must be assigned a value from its domain.

solution: A complete assignment that satisfies every constraint.

Once you have all three, a **(complete) assignment** is a choice of one value, drawn from its own domain, for every single variable in the problem – no variable left unassigned. Not every complete assignment is acceptable, though: a **solution** is a complete assignment that additionally satisfies every constraint in the problem. "Solving" a CSP, in the most basic sense, means finding at least one solution, finding all of them, or proving that none exists.

Definition: Constraint Satisfaction Problem

A CSP is a triple (X, D, C) where $X = \{X_1, \ldots, X_n\}$ is a finite set of variables, $D = \{D_1, \ldots, D_n\}$ assigns each variable X_i a finite domain D_i, and C is a finite set of constraints, each restricting the joint values a subset of X may take. A solution is a complete assignment $X_i \mapsto v_i \in D_i$ for every i such that every constraint in C is satisfied.

Analogy

Picture a paper form with blanks and fine print. Each blank on the form is a variable; the set of entries the form will accept in that blank is its domain (a date field only accepts dates, a yes/no field only accepts two values); and the form's fine print – "field 4 must be later than field 2" – is a constraint. A correctly filled-out form, one where every blank has an entry and every rule in the fine print holds, is a solution. A form with a blank left empty is not even a complete assignment; a form that fills every blank but violates the fine print is a complete assignment that is not a solution.

A very common early misconception is treating "complete assignment" and "solution" as synonyms. They are not: completeness is only about covering every variable, while being a solution additionally demands that every constraint holds. A CSP can easily have complete assignments that are not solutions (you filled in every blank, but broke a rule) – what you are searching for is the much narrower set of assignments that are both complete *and* constraint-satisfying.

1.1.2 A tiny worked example: the triangle map

Before reaching for any classic benchmark problem, walk through the smallest instance that still has all three ingredients. Imagine three regions on a toy map, called A, B and C, where every pair of regions shares a border – so no two of them may be given the same colour, and there are three colours available.

Case Study 1.1: The triangle map, worked by hand

Variables: one per region – A, B, C.
Domain: each variable may take one of three colours, written as integers $\{0, 1, 2\}$ for now (0 = red, 1 = green, 2 = blue).
Constraints: because every pair of the three regions is adjacent, three constraints hold: $A \neq B$, $B \neq C$, $A \neq C$.
One solution: $A = 0$, $B = 1$, $C = 2$ is a complete assignment (every variable has a value from its domain) and satisfies all three constraints ($0 \neq 1$, $1 \neq 2$, $0 \neq 2$), so it is a solution. $A = 0$, $B = 0$, $C = 1$ is a complete assignment too – but it violates $A \neq B$, so it is *not* a solution.

This triangle instance is exactly the toy problem `kod_ornekleri/hafta01/modeller/csp_basics.py` builds and solves in code; Unit 4 shows you the working solver call, but the three ingredients you just wrote by hand – three variables, one shared domain, three inequality constraints – are already the entire model. Later this week (Unit 3), the same three-part recipe scales up to the seven-region Australia map with a real adjacency structure, shown in Figure 1.1: one node per region variable, one edge per "must differ" constraint, filled with a real, solver-verified 3-colouring.

1.1.3 CSP vs constraint optimization problem (COP)

constraint optimization problem (COP): A CSP extended with an objective function; the goal is the best feasible solution, not any feasible solution.

A plain CSP only ever asks one of three questions: is there a solution, find one, or find all of them. Nothing in the definition above ranks solutions against each other – every solution is equally acceptable. A **constraint optimization problem (COP)** keeps the exact same variables, domains and constraints, but adds one more ingredient: an objective function to minimize or maximize, so the question becomes "find the *best* feasible solution," not just any one.

Analogy

Contrast "find a valid travel itinerary" with "find the cheapest valid travel itinerary." Both respect the same rules – a connecting flight must leave after you land. The first only asks for any itinerary that works; the second asks for the best one by some measure. The rules never changed; only the question asked of them did.

This course stays in pure-CSP territory this week, and every problem you model here asks "find a solution" or "find all solutions," never "find the best one." Constraint optimization problems return properly in Week 8, once you have enough CSP machinery under you to appreciate what an objective function adds on top of it.

1.1.4 CSP vs Boolean satisfiability (SAT)

Boolean satisfiability problem (SAT): The special case of a CSP where every variable is Boolean and constraints are logical clauses.

If you have taken a course touching propositional logic, you may already know the **Boolean satisfiability problem (SAT)**: given a Boolean formula, is there an assignment of true/false to its variables that makes the whole formula true? SAT is not a different kind of problem from a CSP – it is a CSP with two restrictions imposed on the general definition above: every domain is forced down to exactly two values, $\{0, 1\}$ (true/false), and every constraint is forced into the shape of a logical clause.

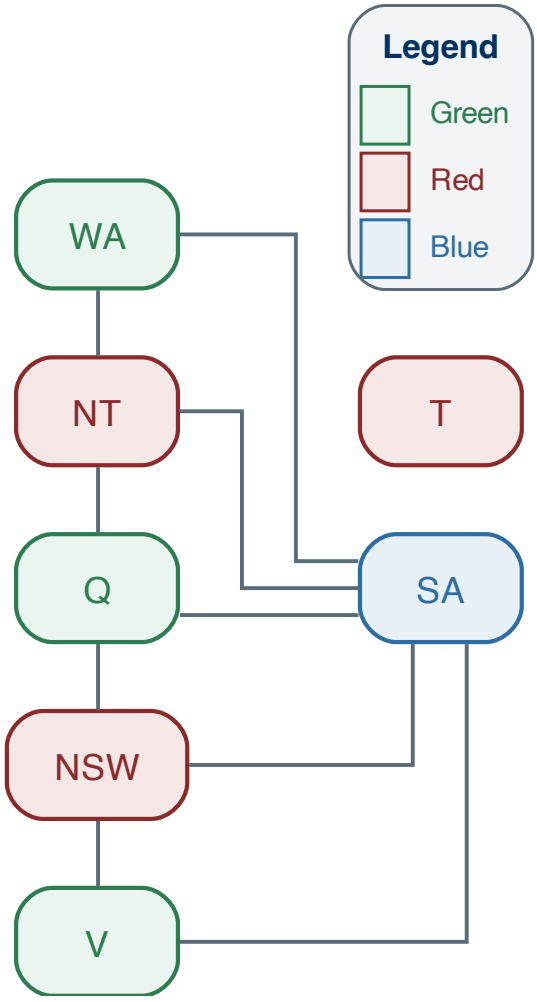


Figure 1.1: Constraint graph for the Australia map-coloring CSP, coloured with a solver-verified 3-colour solution.

A general CSP variable can have any finite domain (colours, time slots, integers 1 to 9), and a general CSP constraint can be any relation at all, not just a clause – so SAT sits strictly inside the wider CSP family. This course does not use a SAT solver this week; the point of the comparison is only to place CSP among its closest relatives before you start building models.

3 • Participation **Now You Try**

A small conference has three talks (T_1, T_2, T_3) and two time slots (morning, afternoon). No two talks that share an author may be scheduled in the same slot; T_1 and T_2 share an author. Write down the variables, one plausible domain, and the one constraint this scenario implies, and say whether finding *any* valid schedule makes this a CSP or a COP.

Feedback. Variables: one per talk – T_1, T_2, T_3. Domain: $\{\text{morning}, \text{afternoon}\}$ for each variable. Constraint: $\text{value}(T_1) \neq \text{value}(T_2)$, since they share an author; T_3 carries no constraint here. As stated – any valid schedule is acceptable – this is a pure CSP; it would only become a COP if an objective were added on top, for example "minimize the number of afternoon slots used."

Check Yourself 1.1

A furniture store's floor plan has 4 display islands arranged so that some pairs are adjacent (they share a walking aisle); no two adjacent islands may show the same product category. Identify the variables, one plausible domain, and one constraint for this scenario, and say whether the task as stated is a CSP or a COP.

5 • Follow-through **Transfer — Where Else?**

The variable/domain/constraint vocabulary you just used is not specific to talks and time slots – it describes every problem this course touches. Weeks 2 through 6 reuse this exact same three-part description for N-queens, Sudoku and map coloring, just with propagation, search and multiple solvers layered on top of the same three ingredients; you will not need to relearn what a domain is again.

1.2 Modeling as a Skill

1 • Pre-instruction **Why It Matters**

Two students model the same magic square puzzle for homework. One's solver finds every solution in under a second; the other's is still running five minutes later – for the exact same problem, on the exact same computer. The difference was never the problem; it was made before either of them typed a single line of solver code, in the choices this unit is about.

1 • Pre-instruction **Prior Knowledge**

Unit 1 gave you the formal CSP triple: variables, domains, constraints. This unit does not add a fourth ingredient to that triple – it teaches you how to choose

good values for the first three before you commit to a model.

It is tempting to think of "modeling" as a mechanical step: read a problem statement, write down its variables, write down its domains, write down its constraints, done. Unit 1's triangle map made that step look almost too easy. But the two students in the opening story wrote down *correct* models of the same magic square – both would have been marked right on paper – and still ended up with wildly different running times once each handed their model to a solver. That gap is the entire subject of this unit: building a good model is a skill you practice and improve, not a transcription exercise you finish once you know the syntax. Three ingredients drive that gap – viewpoint, auxiliary variables, and implied constraints – and this unit walks through each in turn.

1.2.1 Viewpoints: more than one valid way to see the same problem

viewpoint: A particular choice of which variables represent a problem's unknowns.

Take the 4-queens problem: place 4 queens on a 4×4 board so that no two attack each other. Before you can write down variables at all, you have to decide *what* a variable in your model represents – a choice called a **viewpoint** – and there is more than one reasonable answer.

One choice: since exactly one queen must sit in each row (four queens, four rows, no two queens sharing a row), let one variable represent each row, and let its value be the column that row's queen occupies. That gives you 4 variables, each with domain $\{1, 2, 3, 4\}$ – compact, and it happens to make "no two queens share a row" automatic (there is literally one variable per row, so it cannot be violated).

A different, equally valid choice: let one Boolean variable represent each of the 16 squares on the board, taking value 1 if a queen sits there and 0 otherwise. That gives you 16 variables, each with domain $\{0, 1\}$ – bulkier, and "no two queens share a row" now needs an explicit constraint (exactly one of each row's four Booleans is 1) instead of getting it for free. Figure 1.2 places these two choices side by side.

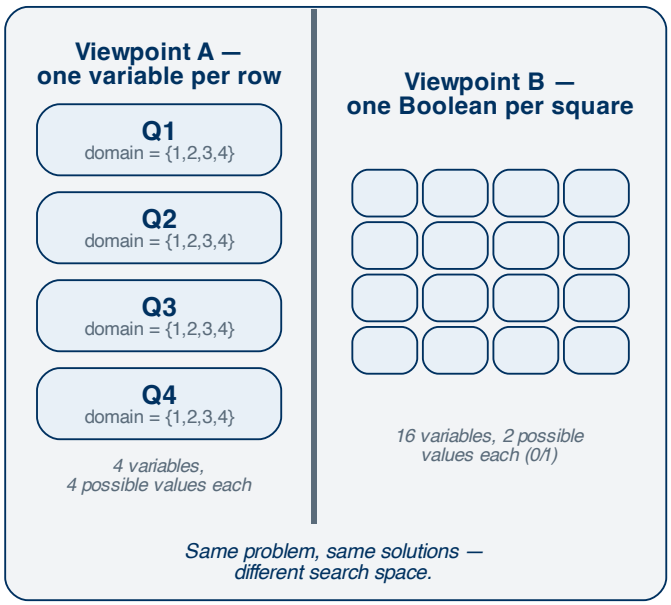


Figure 1.2: Two competing viewpoints for the same 4-queens instance.

Definition: Viewpoint

A particular choice of which variables represent a problem's unknowns. The same problem can have more than one valid viewpoint; none is inherently "wrong," but different viewpoints trade off variable count, domain size, and how naturally later constraints fall out of the choice.

Analogy

Think about describing the same exam timetable two different ways: "one variable per exam, holding its time slot" and "one variable per time slot, holding the set of exams scheduled then" both fully describe the same timetable. Neither description is wrong, but one may make a given constraint – "no student has two exams at the same time" – much easier to write down than the other.

Neither of the two 4-queens viewpoints above is "the correct one." The one-variable-per-row viewpoint is the one this course uses throughout (Unit 3 formalizes it, Unit 4 codes it), precisely because it keeps the variable count low and gets row-uniqueness for free – but the choice was made, not forced. A common misconception at this point is assuming that a problem statement dictates a single, obvious set of variables. It does not: the problem statement dictates what must be true of a *solution*; it is silent on how you represent the unknowns that lead you there, and that silence is exactly where this unit's skill lives.

1.2.2 Auxiliary variables: bookkeeping that is not part of the answer

Once you have committed to a viewpoint, you sometimes find that a constraint you need to state is awkward to write directly in terms of your existing variables. Rather than force it, you can add an **auxiliary variable** – an extra variable added purely to make that one constraint easier to express.

auxiliary variable: A variable added to a model to make some constraint easier to state, without changing what counts as a solution to the original problem.

Definition: Auxiliary variable

A variable added to a model to make some constraint easier to state, without changing what counts as a solution to the original problem.

Analogy

A budget spreadsheet's real inputs are the individual expense cells; a subtotal cell is not a new expense, it is a bookkeeping variable that makes a later rule – "subtotal must stay under the department cap" – easy to check without re-adding every expense cell each time you need that total.

The key phrase in the definition is "without changing what counts as a solution to the original problem." An auxiliary variable is not a new unknown the real-world problem asked you to decide; it is derived from the variables that already answer the problem, and it exists only to give some constraint a direct handle instead of forcing that constraint to re-derive the same information from scratch every time it is checked. A common mistake is worrying that adding variables to a model somehow makes the problem "bigger" or changes its answer – it does not, as long as the auxiliary variable is properly tied (via a constraint) to the variables it summarizes; the set of solutions to the original problem is untouched.

1.2.3 Implied (redundant) constraints: free information for the solver

implied (redundant) constraint: A constraint that logically follows from the constraints already in a model; adding it changes no solution but may help a solver prune faster.

The third ingredient works in the opposite direction from an auxiliary variable: instead of adding a new variable, you add a new **implied (redundant) constraint** – a constraint that was already true all along.

Definition: Implied (redundant) constraint

A constraint that logically follows from the constraints already in a model; adding it changes no solution but may help a solver prune faster.

Case Study 1.2: An implied sum constraint on a permutation

Suppose a model already forces `AllDifferent` on four variables whose shared domain is $\{1, 2, 3, 4\}$ – so any solution assigns them some permutation of 1, 2, 3 and 4. It follows, with no further reasoning needed, that those four variables must sum to $1 + 2 + 3 + 4 = 10$ in *every* solution. Stating `sum(vars) == 10` explicitly changes nothing about which assignments count as solutions – it was already guaranteed by `AllDifferent` – but handing a solver this extra, tighter piece of information can let it rule out an inconsistent partial assignment earlier than the `AllDifferent` constraint alone would have.

Analogy

An implied constraint works like a sanity-check total at the bottom of an exam answer sheet: if every question's points must sum to 100, and you have already listed each question's points correctly, writing "total = 100" at the bottom changes nothing about which answer sheets are correctly filled in – but it lets a grader catch an inconsistency at a glance instead of re-adding every line by hand.

A frequent misconception here is that an implied constraint must add solving power because it is "extra information." It never changes the solution set – by definition, it followed from what was already there – but whether it actually speeds up solving depends on whether it gives the solver's internal propagation a tighter, more directly usable signal than the original constraints did. Sometimes it helps a great deal; sometimes it barely matters. You will not be able to measure that effect precisely until Week 2 introduces propagation – for now, treat implied constraints as a modeling technique you have available, not a guaranteed speedup.

1.2.4 Why this matters before you write a single line of solver code

Put the three ingredients together and you get this unit's central claim: two people modeling the same problem – same variables and domains available to choose from, same constraints to eventually state – can end up with models that behave very differently once you hand them to a solver. Weeks 2 through 6 will make that difference measurable, in propagation strength and running time, on the exact same benchmark problems this week introduces. The choices in this unit – which viewpoint, which auxiliary variables, which implied constraints – are *why* that later difference exists at all; nothing about the solver itself changes between the two students' magic square attempts, only the model each of them handed to it.

3 • Participation **Now You Try**

For Sudoku, one natural viewpoint uses one variable per cell (81 variables, domain 1–9). Propose one *auxiliary* variable you could add to this model to make the constraint "every digit 1–9 appears somewhere in each 3×3 box" easier to check or state, without changing what counts as a solved Sudoku.

Feedback. One reasonable answer is a per-box, per-digit Boolean auxiliary variable `used[box][digit]`, forced equal to "digit `digit` appears somewhere in box `box`." The original 81 cell variables still fully determine a solution on their own – `used[box][digit]` adds no new unknown to the real puzzle; it simply gives the box-constraint a direct handle instead of re-deriving "does this box contain digit d" from the nine cell variables every single time the constraint is checked.

Check Yourself 1.2

Why can adding an implied constraint to an already-correct model make a solver faster, even though the constraint removes no solution?

5 • Follow-through **Transfer — Where Else?**

Every model you build for the rest of this course – scheduling in Weeks 11–12, routing in Week 14 – starts with exactly this choice: which viewpoint, which auxiliary variables, which implied constraints. The problem pool's N-queens, Sudoku and map-coloring instances that Unit 3 formalizes and Unit 4 codes are this course's first, lowest-stakes place to practice that choice before it matters for a harder problem.

1.3 Classic Problems as Models

1 • Pre-instruction **Why It Matters**

N-queens, Sudoku and map coloring are not this course's invention – they are the field's shared benchmark problems, reused in nearly every constraint-programming paper and textbook you will read this semester (and three more times before Week 6). Learning to see them as CSPs, on paper, is what lets you read that literature and immediately recognize: "oh, that is just a CSP with this variable choice."

1 • Pre-instruction **Prior Knowledge**

Unit 1 gave you the formal CSP triple, and Unit 2 gave you three modeling choices – viewpoint, auxiliary variables, implied constraints. This unit is the first place you apply both together, deliberately, to problems you did not invent yourself.

Each problem below gets the same treatment: state its variables, state their domain, state its constraints – exactly the triple Unit 1 defined – and, where a choice exists, name the viewpoint being used, the way Unit 2 taught you to think about

it. Three of the four problems below return as running code in Unit 4; the fourth, magic square, stays on paper this week on purpose.

1.3.1 N-queens

N-queens problem: The classic CSP of placing N queens on an N×N board so that no two share a row, column or diagonal.

The **N-queens problem** asks you to place N queens on an $N \times N$ chessboard so that no two attack each other – meaning no two may share a row, a column, or either diagonal. This course uses the same viewpoint Unit 2 introduced for 4-queens throughout: one variable per row, its value the column of that row's queen.

- **Variables:** $Q_1, \ldots, Q_N$, one per row.
- **Domain:** $\{1, \ldots, N\}$ (the column) for each Q_i.
- **Constraints:** `AllDifferent`$(Q_1, \ldots, Q_N)$ rules out any two queens sharing a column – row-sharing is already impossible because the viewpoint assigns exactly one queen per row, so it needs no separate constraint. Diagonal-sharing needs one more condition: for every pair $i \neq j$, $|Q_i - Q_j| \neq |i - j|$ (two queens i rows apart that also sit i columns apart lie on a shared diagonal, in either direction).

A very common mistake at this point is assuming `AllDifferent` alone solves N-queens – it only rules out shared columns (and, by the viewpoint's construction, shared rows come for free); without the diagonal condition, a solver would happily place two queens on the same diagonal.

Case Study 1.3: Reading a real 8-queens solution

Figure 1.3 shows one genuine, solver-verified solution for $N = 8$ – read directly off a real run of `kod_ornekleri/hafta01/modeller/nqueens.py`, not invented by hand. Notice the figure numbers both rows and columns starting at 1 (a natural convention for a chessboard diagram); Unit 4's code prints the same solution with columns numbered starting at 0, since that is Python's natural indexing for a list. The two numbering conventions describe the identical board – shift every printed column by one to translate between them – and Unit 4 makes this shift explicit again once you see the raw solver output.

1.3.2 Sudoku

Sudoku: The classic CSP of filling a 9×9 grid, with some cells pre-filled, so every row, column and 3×3 box contains each digit 1–9 exactly once.

Sudoku presents a 9×9 grid with some cells already filled in (the *givens*); the task is to fill every remaining cell with a digit 1–9 so that every row, every column, and every one of the nine non-overlapping 3×3 boxes contains each digit exactly once.

- **Variables:** one per cell – 81 variables in total, the viewpoint Unit 2 already used as its running example.
- **Domain:** $\{1, \ldots, 9\}$ for every cell variable.
- **Constraints:** each given fixes its cell's variable by equality (`cell == value`); `AllDifferent` applies to each of the 9 rows, each of the 9 columns, and each of the 9 3×3 boxes – 27 `AllDifferent` constraints in total, each over 9 variables.

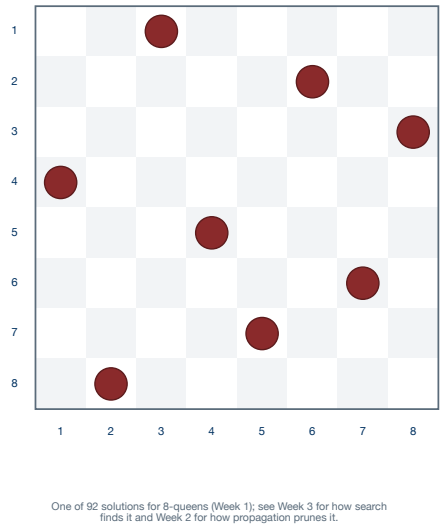


Figure 1.3: One of 92 solutions for 8-queens.

Notice that a Sudoku's givens are themselves constraints, not a separate kind of ingredient – an equality `cell == value` is just as much a constraint, in the formal sense of Unit 1, as `AllDifferent` is; a puzzle with zero givens would still be a well-formed CSP (an extremely under-constrained one, with many solutions), it would simply be a strange one to hand a person to solve by hand.

1.3.3 Map coloring

map coloring problem: The classic CSP of assigning each region of a map a colour, from a fixed palette, so that no two adjacent regions share a colour.

The **map coloring problem** takes a map's adjacency structure – which regions share a border – together with a fixed number of available colours, and asks for an assignment of one colour to each region so that no two adjacent regions share a colour.

This unit's worked instance, reused again in Unit 4, is the classic Australia map from Russell and Norvig's *AIMA* textbook: seven regions – Western Australia (WA), Northern Territory (NT), South Australia (SA), Queensland (Q), New South Wales (NSW), Victoria (V), and Tasmania (T) – with adjacency edges WA–NT, WA–SA, NT–SA, NT–Q, SA–Q, SA–NSW, SA–V, Q–NSW and NSW–V. Tasmania is an island; it borders no other region on this map at all – a deliberately included edge case, since a variable can carry zero constraints and still need a domain value assigned to it (Tasmania still needs a colour, it just has no adjacency rule to satisfy while choosing one).

- **Variables:** one per region – WA, NT, SA, Q, NSW, V, T.
- **Domain:** 3 colours for every region.
- **Constraints:** one $\neq$ constraint per adjacency edge – nine constraints in total, one for each pair of regions listed above.

Figure 1.1 draws exactly this structure as a *constraint graph*: one node per region variable, one undirected edge per $\neq$ constraint, filled with a real, solver-verified 3-colouring read from a genuine run of `kod_ornekleri/hafta01/modeller/map_coloring.py`. Notice Tasmania sits in the figure with no edge touching it at all, exactly as the adjacency list above says it should.

Unlike N-queens and Sudoku, map coloring's constraints use no `AllDifferent` at all – adjacency here is not an "every pair differs from every other pair" relation

across the whole variable set (WA and Tasmania are not adjacent, so nothing forces them to differ); it is a relation defined edge by edge, which is exactly why the model uses one pairwise $\neq$ per adjacency edge instead of one global constraint over every region at once.

1.3.4 Magic square (paper only this week)

magic square: The classic CSP of arranging the numbers 1 to N² in an N×N grid so that every row, column and main diagonal sums to the same value.

The **magic square** problem arranges the numbers 1 through N^2 in an $N \times N$ grid, each number used exactly once, so that every row, every column, and both main diagonals sum to the same value.

- **Variables:** one per cell – N^2 variables.
- **Domain:** $\{1, \ldots, N^2\}$ for every cell.
- **Constraints:** `AllDifferent` over all N^2 cells (every number used exactly once), plus one equal-sum constraint per row, per column, and per diagonal.

This problem is formulated on paper only this week – no code excerpt appears for it, and it is a deliberate choice, not an oversight: Week 3 needs a problem with a larger, harder search space to make search heuristics visible, and magic square's search space grows fast enough with N to serve that purpose well. Its code treatment starts there, not here.

3 • Participation **Now You Try**

Formulate the 4-queens problem on paper using the "one queen per row" viewpoint: state the variables, the domain, and the constraints.

Feedback. Variables: Q_1, Q_2, Q_3, Q_4 (one per row). Domain: $\{1, 2, 3, 4\}$ (the column) for each variable. Constraints: `AllDifferent`(Q_1, Q_2, Q_3, Q_4) (no two queens share a column) and, for all $i \neq j$, $|Q_i - Q_j| \neq |i - j|$ (no two queens share a diagonal) – row uniqueness needs no separate constraint, because the viewpoint already assigns exactly one queen to each row.

Check Yourself 1.3

A classmate formulates 4-queens with the "one queen per row" viewpoint using variables Q_1, Q_2, Q_3, Q_4, domain $\{1, 2, 3, 4\}$ each, and only the constraint `AllDifferent`(Q_1, Q_2, Q_3, Q_4). Identify what is missing from this formulation, and give a concrete assignment that satisfies this single constraint yet is not a valid 4-queens solution.

5 • Follow-through **Transfer — Where Else?**

The paper models you just wrote for N-queens, Sudoku and map coloring are not an exercise you leave behind – Unit 4 turns them directly into running CP-SAT code today, and Weeks 2 (propagation), 3 (search) and 6 (three solvers) will keep coming back to these exact same three problems, so the modeling work you just did keeps paying off well beyond this week.

1.4 First Models in OR-Tools CP-SAT

1 • Pre-instruction **Why It Matters**

Everything so far – the formal triple, the modeling choices, the paper formulations – has been on paper. This unit is where you finally get to watch a computer actually place 8 queens on a board with no two attacking each other, in a fraction of a second, from a model you can read and understand line by line.

1 • Pre-instruction **Prior Knowledge**

Unit 3's paper CSP formulations for N-queens, Sudoku and map coloring are this unit's starting point; this unit's code is a direct, line-for-line translation of those variables, domains and constraints, not a new model.

OR-Tools CP-SAT: Google's constraint-programming solver, used in this course through its Python interface, the `ortools` package's CP-SAT module.

This course's primary tool is **OR-Tools CP-SAT**, Google's open-source constraint-programming solver, used through its Python interface, the `cp_model` module of the `ortools` package. Every model you build this semester – CP-SAT this week, and MIP solvers such as Gurobi from Week 7 onward – follows the same broad recipe: create a model object, declare decision variables with their domains, add constraints, then ask a solver to search for an assignment. This unit walks that recipe through exactly once, on the toy CSP from Unit 1, then again on each of Unit 3's three coded problems.

1.4.1 The three building blocks

Three pieces of CP-SAT's API cover almost everything this course does with it. First, a `cp_model.CpModel()` instance – an empty container you populate with variables and constraints, corresponding directly to the (X, D, C) triple from Unit 1. Second, decision variables, created with `model.NewIntVar(low, high, name)` – each call declares one variable whose domain is every integer from `low` to `high` inclusive. Third, constraints, added either with the general-purpose `model.Add(...)` (for anything expressible as a Python comparison, such as `model.Add(x != y)`) or with a dedicated **global constraint** method such as `model.AddAllDifferent([...])` – the code form of the **AllDifferent** relation Unit 2 and Unit 3 already used on paper.

global constraint: A constraint over an entire group of variables at once, expressing a relation that would otherwise need many pairwise constraints.

The toy triangle CSP from Unit 1 makes all three blocks concrete before any classic problem's complexity gets in the way. You declare its three region variables exactly like this:

```
region_a = model.NewIntVar(0, 2, "region_a")
region_b = model.NewIntVar(0, 2, "region_b")
region_c = model.NewIntVar(0, 2, "region_c")
return [region_a, region_b, region_c]
```

Each `NewIntVar(0, 2, ...)` call is Unit 1's domain $\{0, 1, 2\}$ written directly into code – three colours, numbered from 0. Its three pairwise-inequality constraints – one per edge of the triangle, exactly as Unit 1 stated them by hand – follow the same direct translation:

```
model.Add(region_a != region_b)
model.Add(region_b != region_c)
model.Add(region_a != region_c)
```

Common Pitfall

A frequent mistake when moving from a paper model to CP-SAT code is forgetting that `model.Add(...)` does not *evaluate* a Python comparison – it *records* one. Writing `region_a != region_b` outside a `model.Add(...)` call does nothing useful (Python happily evaluates it against the `IntVar` objects themselves, not their eventual solved values, and throws the result away); the comparison only becomes part of the CSP once it is wrapped in `model.Add(...)` and attached to the model.

1.4.2 Solving once

With variables and constraints declared, solving is a fixed three-line pattern: create a `cp_model.CpSolver()`, call `solver.Solve(model)`, and read the returned status against CP-SAT's named constants – `cp_model.OPTIMAL` or `cp_model.FEASIBLE` both mean a **feasible** CSP was found (this week's problems carry no objective, so `OPTIMAL` and `FEASIBLE` both simply mean "a solution exists and here it is"), while `cp_model.INFEASIBLE` means the solver proved the CSP infeasible – no solution exists at all. Once solved, `solver.Value(var)` reads the value CP-SAT assigned to any given variable in the found solution:

feasible: A CSP is feasible if at least one solution exists, infeasible if none does.

```python
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
values = [solver.Value(region) for region in regions]
print(f"Status: {solver.StatusName(status)}")
print(f"Assignment: region_a={values[0]}, region_b={values[1]}, region_c={
    values[2]}")
```

Expected Output

```
Status: OPTIMAL
Assignment: region_a=2, region_b=1, region_c=0
```

This is the real, captured output of the code above – a status of `OPTIMAL` and one valid three-colouring of the triangle, with `region_a`, `region_b` and `region_c` all pairwise distinct, exactly as Unit 1's three constraints require. A **constraint solver** such as CP-SAT never shows you the search it performed to reach this answer – Week 3 opens that box; this week, the solver is a black box that simply returns a solution.

constraint solver: Software that, given a CSP's variables, domains and constraints, searches for one or all solutions (or proves none exists).

Analogy

Treat a constraint solver like a lab's room-booking assistant: you hand the assistant the rooms, the time slots, and the rules (no double-booking, a particular lab needs a fume hood), and the assistant searches the possibilities and hands you back a valid schedule – or tells you none exists. You never watch it try and discard the bad combinations one by one; you only see the rules you gave it and the answer it returns.

1.4.3 Enumerating all solutions

`solver.Solve(model)` stops at the very first solution it finds – exactly what you

want when any one valid answer will do, but not enough when the question is "how many solutions does this CSP have." CP-SAT answers that second question with a different mode: set `solver.parameters.enumerate_all_solutions = True` and pass a *solution callback* object – a subclass of `cp_model.CpSolverSolutionCallback` – to `solver.Solve(model, callback)`. CP-SAT then keeps searching after every solution it finds, instead of stopping at the first one, and calls the callback's `OnSolutionCallback` method once for every complete solution it encounters – which is how you count solutions rather than merely find one.

Analogy

Enumerating all solutions is like a scheduling assistant that keeps searching instead of stopping at the first valid timetable it finds: one that stops early answers "can this be done"; one that keeps going and reports every valid timetable it finds answers "how many different ways can this be done" – the second is what a solution count is built from.

1.4.4 N-queens in code

Unit 3's "one queen per row" viewpoint translates into CP-SAT almost unchanged. One integer variable per row, domain `0` to `n-1` (Python's natural 0-indexed columns, in contrast to the 1-indexed columns used on paper and in Figure 1.3 – keep this one indexing shift in mind as you read printed output against the figure):

```
queens = [model.NewIntVar(0, n - 1, f"queen_{row}") for row in range(n)]
```

Unit 3's two constraint groups – `AllDifferent` on columns, plus the diagonal condition – become three `AllDifferent` calls: one on the columns directly, and two more on the columns after Unit 3's diagonal transformations ($Q_i + i$ and $Q_i - i$), since two queens share a diagonal exactly when one of these two transformed values coincides for both of them:

```
model.AddAllDifferent(queens)
model.AddAllDifferent([queens[i] + i for i in range(n)])
model.AddAllDifferent([queens[i] - i for i in range(n)])
```

Solving once, for $N = 8$, and printing the result column by column produces this real captured output:

Expected Output

```
Status: OPTIMAL
Columns per row (0-indexed) for N=8: [2, 5, 7, 0, 3, 6, 4, 1]
```

Figure 1.3 draws this exact solution – row 1's queen sits at 0-indexed column 2, drawn as 1-indexed column 3 on the board, and so on down the list, one shift of 1 applied uniformly to every entry. Enumerating instead of solving once, for the smaller $N = 6$ (chosen so the full count is easy to sanity-check by hand later in the course), gives:

Expected Output

```
Number of solutions for N=6: 4
```

Four is the well-known solution count for 6-queens (OEIS A000170) – a fact the course's automated tests check independently, precisely so that a wrong number

here would be caught as a bug, never quietly accepted as "the number this week happened to produce."

1.4.5 Sudoku and map coloring in code

Sudoku's translation follows the same pattern: 81 cell variables, domain 1–9, the puzzle's givens applied by equality, and `AllDifferent` over every row, column and 3×3 box exactly as Unit 3 stated on paper. Running `kod_ornekleri/hafta01/modeller/sudoku.py` against one fixed puzzle produces a fully completed, solver-verified grid – the real captured output lives in `kod_ornekleri/hafta01/ciktilar/sudoku-solution.txt`, and every row, column and box in it is a permutation of 1 through 9, exactly as the model's constraints demand.

Map coloring's translation is the one place this week where `AllDifferent` is deliberately absent – Unit 3 already explained why: adjacency is an edge-by-edge relation, not an everyone-differs-from-everyone-else relation over the whole variable set. The code adds one pairwise `!=` per adjacency edge, directly mirroring Unit 3's nine constraints, and its real captured output – the one Figure 1.1 draws – lives in `kod_ornekleri/hafta01/ciktilar/mapcoloring-solution.txt`.

3 • Participation **Now You Try**

A partially written CP-SAT script for 6-queens is missing its diagonal constraints (only `model.AddAllDifferent(queens)` for columns is present). Add the two missing `AddAllDifferent` calls needed to also forbid shared diagonals, then run the script and report the solution count.

Feedback. The two missing calls are `model.AddAllDifferent([queens[i] + i for i in range(6)])` (no shared "/" diagonal) and `model.AddAllDifferent([queens[i] - i for i in range(6)])` (no shared "\" diagonal). With both added, enumerating all solutions for $N = 6$ reports exactly **4** solutions – the well-known 6-queens count (OEIS A000170), matching the real output shown above.

Check Yourself 1.4

In OR-Tools CP-SAT, what is the difference between calling `solver.Solve(model)` once and enumerating solutions with `solver.parameters.enumerate_all_solutions = True` plus a solution callback? When do you need the second one?

5 • Follow-through **Transfer — Where Else?**

The three scripts you completed this week – N-queens, Sudoku, map coloring – are not disposable homework: Week 2 takes the exact same N-queens and map-coloring models and asks how much work the solver's propagation actually saved you, Week 3 asks which variable and value order found this solution fastest, and Week 6 runs the same model on two more solvers entirely. Keep these files; you will open them again.

Chapter Summary

Summary

This chapter gave you the vocabulary the rest of this course is built on. Unit 1 defined a constraint satisfaction problem as a triple of variables, domains and constraints, defined a solution as a complete assignment satisfying every constraint, and placed CSPs among their closest relatives – constraint optimization problems, which add an objective function, and Boolean satisfiability, which restricts every domain to two values. Unit 2 showed that two correct models of the same problem can behave very differently once you hand each to a solver, and gave you three concrete levers behind that difference: viewpoint choice, auxiliary variables, and implied (redundant) constraints. Unit 3 applied both units together to four classic benchmark problems – N-queens, Sudoku, map coloring and, on paper only, magic square – stating each one's variables, domain and constraints explicitly. Unit 4 turned three of those paper models into running OR-Tools CP-SAT code: you declared variables with `NewIntVar`, added constraints with `model.Add` and `AddAllDifferent`, solved once with `solver.Solve`, and enumerated every solution to 6-queens with a solution callback, closing the loop from a formal definition to a verified, running solver call. Next week takes the exact same N-queens and map-coloring models you built here and opens the solver's black box: you will watch, step by step, how propagation shrinks each variable's domain before any search happens, and you will build a second, differently-viewpointed model of the same problem to measure how much this week's modeling choices already mattered.

Check Yourself Answers

Question (Unit 1 – furniture store display islands). Variables: one per display island. Domain: the set of product categories the store carries. Constraint: for every pair of adjacent islands (sharing a walking aisle), their assigned categories must differ. As stated, any valid assignment of categories to islands that respects the adjacency rule is acceptable, so this is a pure CSP; it would only become a COP if an objective were added on top – for example, "minimize the number of category changes a shopper passes while walking the floor."

Question (Unit 2 – why an implied constraint can speed up a solver without removing any solution). The set of solutions is unchanged by definition – an implied constraint follows logically from constraints already in the model, so nothing it states was ever false before it was added. What changes is the information available to the solver's internal propagation: the implied constraint hands it an extra, tighter piece of information that can rule out an inconsistent partial assignment earlier in the search than the original constraints alone would have. Same solution set, often a smaller search tree explored to find it.

Question (Unit 3 – the classmate's incomplete 4-queens formulation). The formulation is missing the diagonal constraints: for all $i \neq j$, $|Q_i - Q_j| \neq |i - j|$. `AllDifferent`(Q_1, Q_2, Q_3, Q_4) alone only rules out two queens sharing a column – row uniqueness is already free from the viewpoint, but nothing yet rules out a

shared diagonal. A concrete counterexample: $Q_1 = 1, Q_2 = 2, Q_3 = 4, Q_4 = 3$ satisfies `AllDifferent` (all four values are distinct), but Q_1 and Q_2 share a diagonal – $|Q_1 - Q_2| = 1$ equals $|1 - 2| = 1$ – so this assignment places two attacking queens on the board and is not a valid 4-queens solution.

Question (Unit 4 – solving once vs enumeration). `solver.Solve(model)` stops at the first solution the search finds, or proves infeasibility if none exists – it never keeps looking once it has one answer. Enumeration mode – setting the solver's `enumerate_all_solutions` parameter to `True` and passing a solution callback – instead invokes the callback once for every complete solution found and keeps the search going afterward, so it is the mode you need whenever the question is not "does a solution exist" but "how many are there, or what are all of them" – exactly the mode this chapter used to confirm 6-queens has precisely 4 solutions, rather than merely producing one example of one.